

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Дисциплина: Бэк-энд разработка

Отчет

**Домашняя работа №4 «Технический дизайн
микросервисной архитектуры»**

Выполнила:

Клапнева Диана

Группа К3342

Проверил:

Добряков Д. И.

Санкт-Петербург

2026 г.

Задача

Задание:

1. Разделите ваше монолитное бэкенд-приложение на микросервисы, придерживаясь концепции database-per-service
2. Спроектируйте микросервисную архитектуру: отразите взаимосвязь между микросервисами, опишите способы их взаимодействия друг с другом
3. Спроектируйте разделение вашей БД на обособленные БД
4. Спроектируйте и опишите эндпоинты, необходимые для обеспечения межсервисного взаимодействия в формате спецификации OpenAPI
5. Опишите варианты запросов и ответов, задокументируйте возможные ошибки
6. В результате вы должны сформировать документ, в котором будут описаны конкретные требования и шаги для достижения разделения вашего монолитного бэкенд-приложения на отдельные микросервисы

Необходимо сделать отчёт по шаблону

Ход работы

1 Бизнес-логика разделения на микросервисы

Всего в системе должно быть 4 микросервиса: система управления аккаунтами и авторизацией пользователей, система управления записями о недвижимости, система управления сделками (изменение их статуса от “в процессе оформления” до завершенной) и система чата. Также нужен сервис, отвечающий за маршрутизацию.

Согласно концепции database-per-service, будет 4 отдельных баз данных: users_db, estates_db, deals_db, messages_db.

2 Микросервисная архитектура

Взаимодействие реализуется через API Gateway: все внешние запросы от клиентов поступают на API Gateway - единую точку входа, которая проверяет аутентификацию, логирует запросы и маршрутизирует их к нужному микросервису (/api/v1/users → User Service, /api/v1/estates → Estate Service и т.д.), при этом Gateway также может агрегировать данные из нескольких сервисов в один ответ, чтобы клиент получал готовые составные данные без множества отдельных запросов.

3 Структура баз данных микросервисов

В целом, структура новых отдельных баз данных такая же, как и в соответствующих таблицах для монолитного приложения.

```
CREATE TABLE users (  
    id SERIAL PRIMARY KEY,  
    first_name VARCHAR(50) NOT NULL,  
    middle_name VARCHAR(50),  
    last_name VARCHAR(50) NOT NULL,  
    deal_role VARCHAR(20) CHECK (deal_role IN ('landlord', 'tenant')),  
    email VARCHAR(255) UNIQUE NOT NULL,  
    password VARCHAR(255) NOT NULL,  
    is_verified BOOLEAN DEFAULT FALSE,  
    created_at TIMESTAMP DEFAULT NOW(),  
    updated_at TIMESTAMP  
);|
```

```
CREATE TABLE estates (  
    id SERIAL PRIMARY KEY,  
    user_id INTEGER NOT NULL,  
    address TEXT NOT NULL,  
    type VARCHAR(20) NOT NULL CHECK (type IN ('apartment', 'cottage', 'room')),  
    room_amount INTEGER NOT NULL,  
    description TEXT,  
    price DECIMAL(10,2) NOT NULL,  
    image_path VARCHAR(255),  
    bath_type VARCHAR(20) CHECK (bath_type IN ('shower', 'bathtub')),  
    fridge BOOLEAN DEFAULT FALSE,  
    washing_machine BOOLEAN DEFAULT FALSE,  
    internet BOOLEAN DEFAULT FALSE,  
    tv BOOLEAN DEFAULT FALSE,  
    furnished_rooms BOOLEAN DEFAULT FALSE,  
    furnished_kitchen BOOLEAN DEFAULT FALSE,  
    is_verified BOOLEAN DEFAULT FALSE,  
    is_available BOOLEAN DEFAULT TRUE,  
    created_at TIMESTAMP DEFAULT NOW(),  
    updated_at TIMESTAMP  
);
```

```
CREATE TABLE deals (  
    id SERIAL PRIMARY KEY,  
    landlord_id INTEGER NOT NULL,  
    tenant_id INTEGER NOT NULL,  
    period VARCHAR(50) NOT NULL,  
    deal_status VARCHAR(20) DEFAULT 'pending' CHECK (deal_status IN ('pending', 'active', 'closed')),  
    is_published BOOLEAN DEFAULT FALSE,  
    created_at TIMESTAMP DEFAULT NOW(),  
    updated_at TIMESTAMP  
);
```

```
CREATE TABLE deal_estates (  
    deal_id INTEGER NOT NULL,  
    estate_id INTEGER NOT NULL,  
    PRIMARY KEY (deal_id, estate_id)  
);|
```

```

CREATE TABLE sessions (
    id SERIAL PRIMARY KEY,
    user_id INTEGER NOT NULL,
    created_at TIMESTAMP DEFAULT NOW(),
    updated_at TIMESTAMP
);

CREATE TABLE messages (
    id SERIAL PRIMARY KEY,
    user_id INTEGER NOT NULL,
    session_id INTEGER NOT NULL REFERENCES sessions(id),
    message TEXT NOT NULL,
    attachment_file_path VARCHAR(255),
    is_resshared BOOLEAN DEFAULT FALSE,
    created_at TIMESTAMP DEFAULT NOW(),
    updated_at TIMESTAMP
);

```

4 Эндпоинты, необходимые для межсервисного взаимодействия

User-service

Метод	URL	Запрос
GET	/api/v1/users	Получить всех пользователей
GET	/api/v1/users/[id]	Получить пользователя по ID
POST	/api/v1/users	Создать пользователя
PATCH	/api/v1/users/[id]	Обновить пользователя
DELETE	/api/v1/users/[id]	Удалить пользователя

Estate-service

Метод	URL	Запрос
GET	/api/v1/estates	Получить все объекты
GET	/api/v1/estates/[id]	Получить объект по ID
POST	/api/v1/estates	Создать объект

PATCH	/api/v1/estates/[id]	Обновить объект
DELETE	/api/v1/estates/[id]	Удалить объект

Deal-service

Метод	URL	Запрос
GET	/api/v1/deals	Получить все сделки
GET	/api/v1/deals/[id]	Получить сделку по ID
POST	/api/v1/deals	Создать сделку
PATCH	/api/v1/deals/[id]	Обновить сделку
DELETE	/api/v1/deals/[id]	Удалить сделку

Messaging-service

Метод	URL	Запрос
GET	/api/v1/sessions	Получить все сессии
GET	/api/v1/sessions/[id]	Получить сессию по ID
POST	/api/v1/sessions	Создать сессию
DELETE	/api/v1/sessions/[id]	Удалить сессию
GET	/api/v1/messages	Получить все сообщения
GET	/api/v1/messages/[id]	Получить сообщение по ID
POST	/api/v1/messages	Создать сообщение
PATCH	/api/v1/messages/[id]	Обновить сообщение
DELETE	/api/v1/messages/[id]	Удалить сообщение

5 Примеры запросов и ответов, возможные коды ошибок

В контроллерах для каждого сервиса должны быть прописаны запросы и возможные ответы на них. Приведем пример с помощью сервиса управления записями недвижимости:

При получении записи объекта может быть ошибка 404 (не найден такой объект) либо 500 (внутренняя ошибка сервера).

При создании новой записи объекта может быть ответ 201 (создано), 422 (ошибка валидации), 500 (внутренняя ошибка сервера).

При удалении записи объекта может быть ответ 204 (удалено), 404 (не найден такой объект), 500 (внутренняя ошибка сервера).

При изменении записи объекта может быть ответ 422 (ошибка валидации), 404 (не найден такой объект), 500 (внутренняя ошибка сервера).

Ровно так же обрабатываются запросы к остальным сервисам. При этом при успешном создании, изменении и получении объекта возвращается json с верными данными.

Вывод

Данная работа – формализованные требования к выполнению лабораторной работы №2 по реализации разделения монолитного бекенд-приложения на отдельные микросервисы. В условиях рабочего проекта такой документ необходим для полноценного планирования разработки.